Sistemas distribuidos

Laboratorio S09

Cortés Bolaños Angel David - 423066940

Seccion 1

El cliente recibe un error HTTP 500 y la transferencia se queda atorada permanentemente en estado ABONADA (a pesar de que el saldo ya se movió) porque la saga estaba acoplada de forma síncrona al servicio de notificación. Al estar apagado dicho servicio, la petición HTTP se queda esperando respuesta hasta arrojar un error de conexión, interrumpiendo el flujo antes de que la base de datos marque la operación como COMPLETADA.

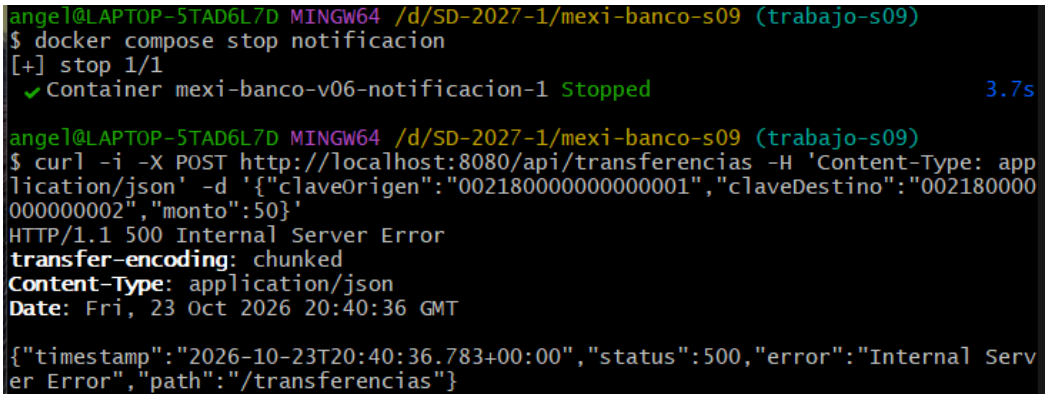


Figura 1. La transferencia arroja error 500 (la transferencia se quedó a medias)

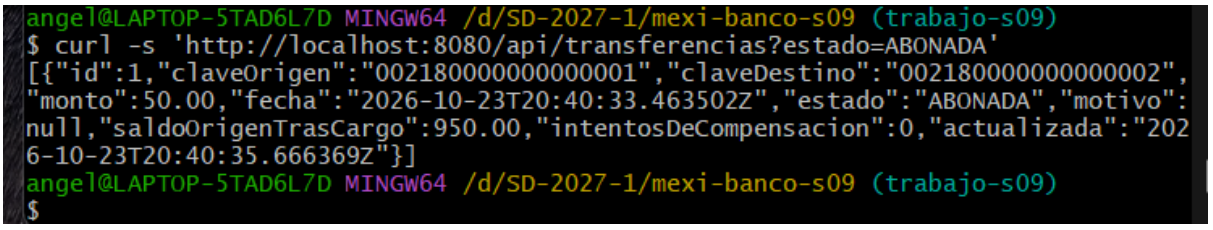


Figura 2. Pero aparece en estado abonada

Sección 2

Al implementar RabbitMQ, el servicio de transferencia deja de depender de una respuesta HTTP en tiempo real. Ahora, primero guarda su estado final en la base de datos y posteriormente delega un recado asíncrono (publicando un evento TransferenciaRealizada) en el exchange transferencias de tipo Topic. Garantía del Publisher Confirm: Activar publisher-confirm-type: correlated garantiza estrictamente que el broker de RabbitMQ recibió el mensaje, lo guardó en disco y asumió la responsabilidad de rutearlo. Lo que no garantiza es que el mensaje haya llegado a una cola específica, ni tampoco evita que el evento se pierda si el microservicio de transferencia sufre una caída fatal milisegundos antes de intentar publicar el evento.

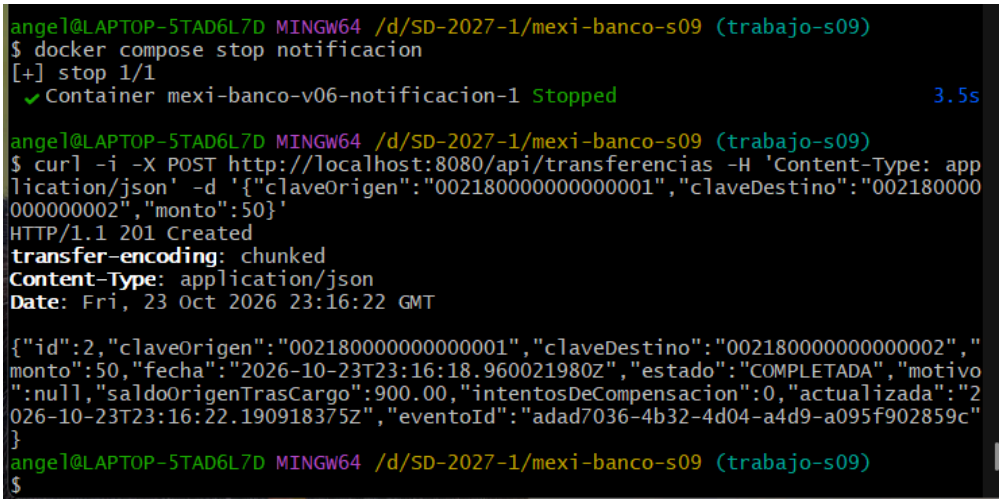


Figura 3. Se logró la transferencia (201) aún con notificación apagado

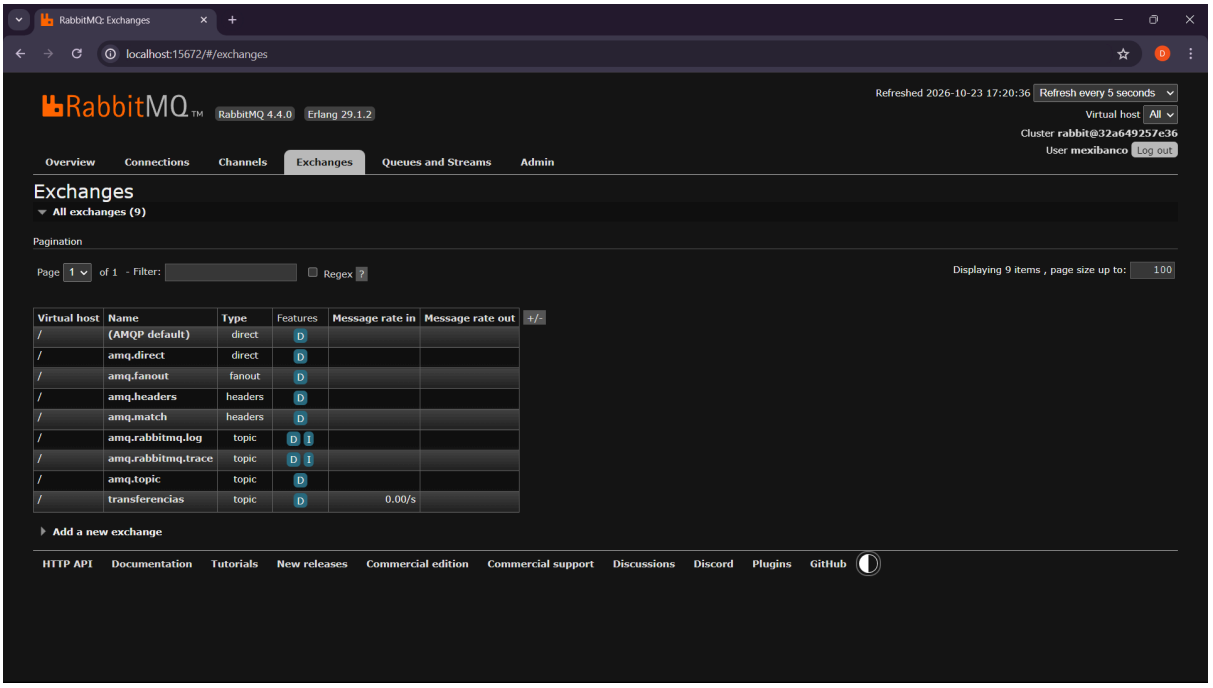


Figura 4.Transferencias tipo Topic y letra D

Sección 3

Se aislaron los microservicios apagando intencionalmente notificacion y ejecutando tres transferencias. El sistema devolvió códigos 201 exitosos y los tres eventos quedaron pacientemente retenidos en la cola notificacion.transferencias (3 mensajes en estado 'Ready'). Al restablecer el servicio, el consumidor procesó los mensajes acumulados en un solo bloque (Figura 7). Esto demuestra el concepto de desacople en tiempo: el productor y el consumidor operan de manera independiente y las caídas de red no generan pérdida de datos. De igual forma, la Figura 8 evidencia la resiliencia de la saga: al simular una falla en el microservicio de cuentas de destino, el sistema compensó (revirtió) automáticamente el saldo extraído inicialmente y emitió el evento TransferenciaRevertida hacia RabbitMQ, garantizando que la cuenta origen (Ana) reciba su notificación de devolución sin inconsistencias financieras.

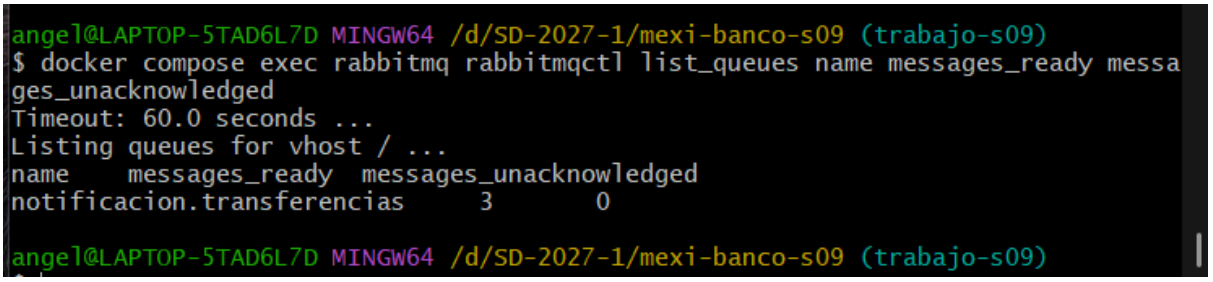


Figura 5. Se ven los mensajes que quedaron encolados

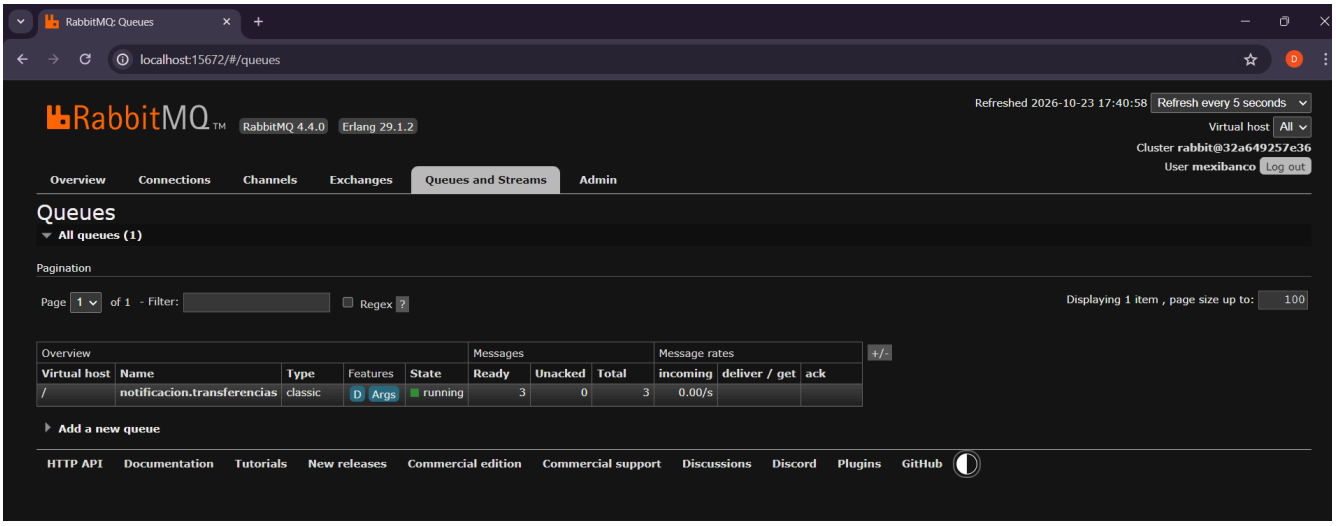


Figura 6.Vista de los mensajes encolados en Rabbit

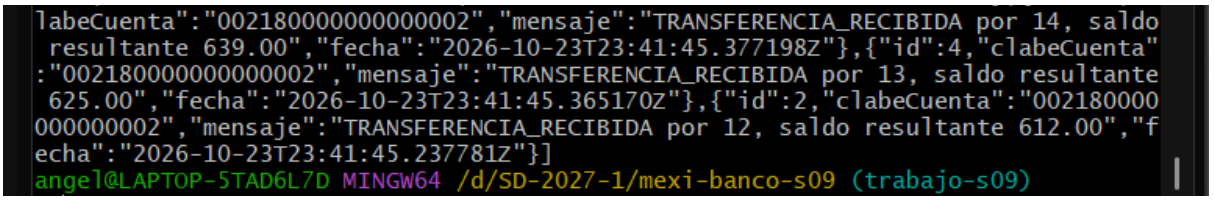


Figura 7. Al encender se observa que se hace las 3 de golpe

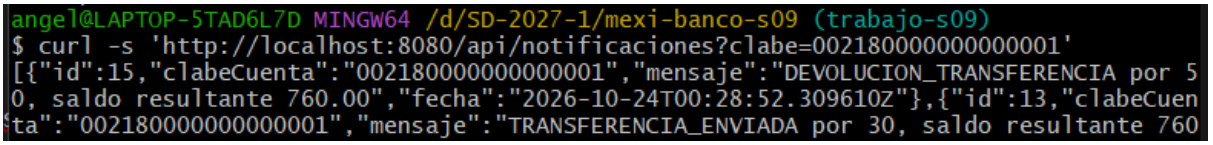


Figura 8. Ana recibe su devolución de transferencia

Sección 4

Debido a que RabbitMQ garantiza la entrega "al menos una vez" (*at-least-once*), forzamos una falla clonando y republicando el mismo payload desde el panel, lo que resultó en avisos duplicados en la base de datos (Figuras 9 y 10).

Solución por Idempotencia: Para erradicar esto, implementamos una tabla evento_procesado controlada por el eventoId único de cada transferencia. Dentro de la misma transacción de base de datos donde se guardan los avisos, insertamos este candado mediante una consulta nativa (ON CONFLICT DO NOTHING). Si el sistema detecta que el ID ya existe, aborta la inserción repetida de los avisos (retornando 0) pero aun así emite el

ack manual al broker para que deseche el clon de la cola. Las Figuras 11 y 12 demuestran la efectividad de este patrón, donde el segundo intento se ignora exitosamente

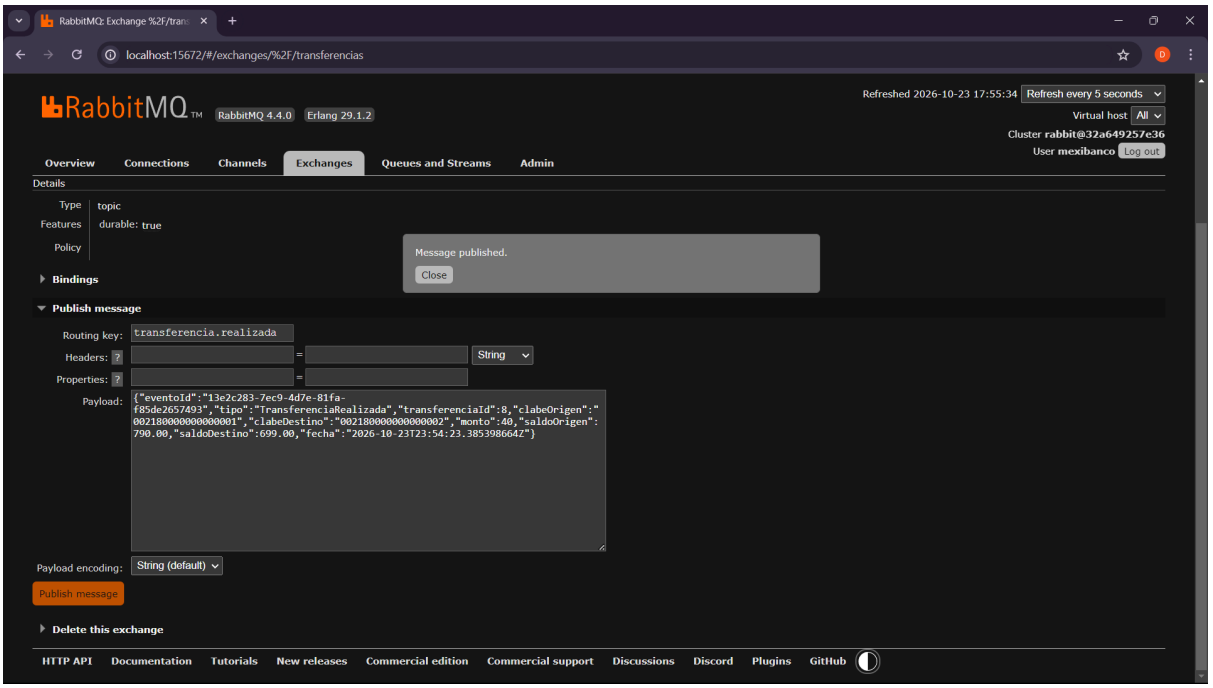


Figura 9. Se hizo una transferencia y se duplicó

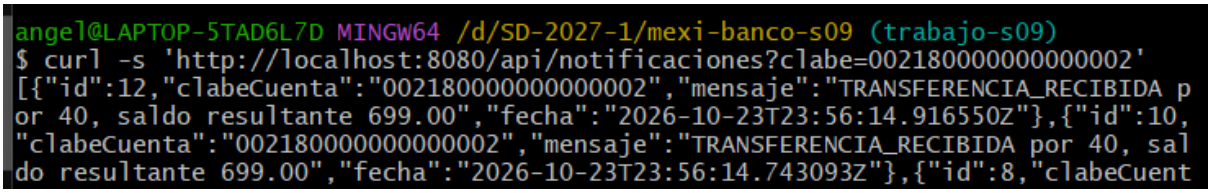


Figura 10. Se puede ver que se duplicó cosa que no se quiere

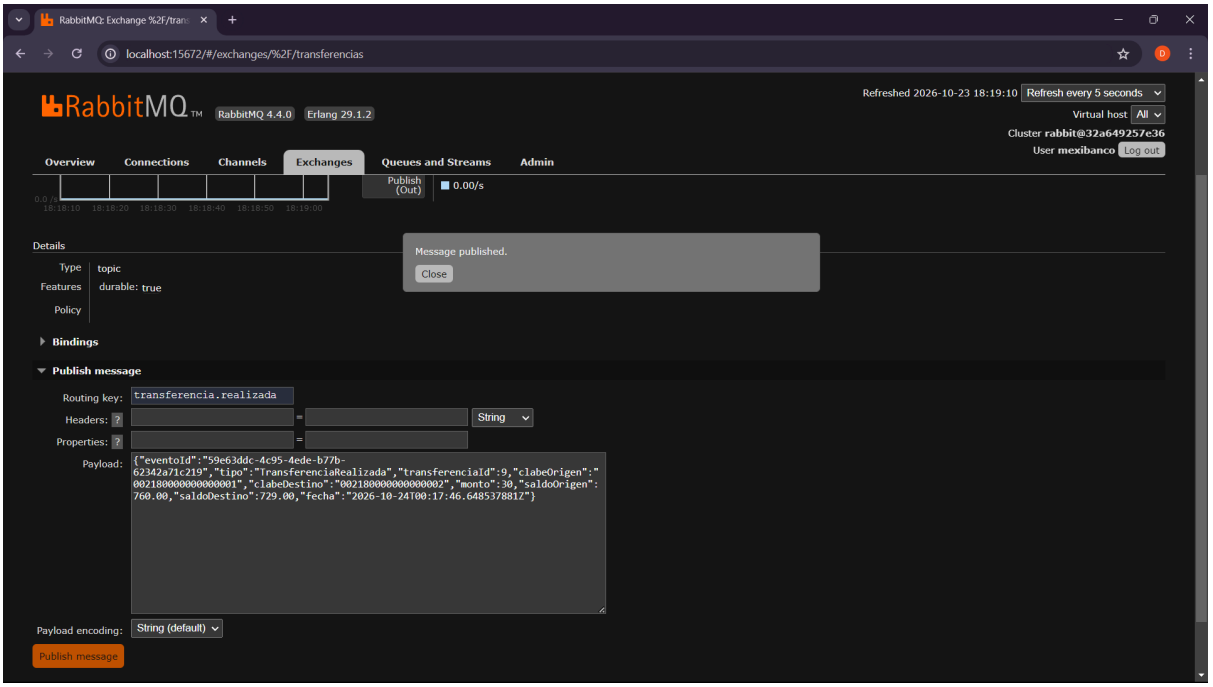


Figura 11. Se genera otra transferencia y se duplica

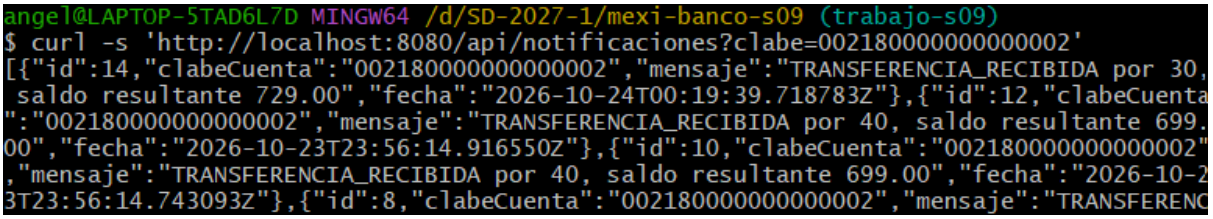


Figura 12. Se puede ver que ya solo se realiza una vez

Sección 5

Al enviar un JSON mal formado (mensaje envenenado), el consumidor fallaba intentando parsearlo repetidamente. Tras configurar la topología DLQ, el consumidor atrapa la excepción y emite un basicNack con requeue=false. Esto ordena a RabbitMQ retirar el mensaje defectuoso de la cola principal y rutearlo hacia transferencias.dlx, dejándolo en la cola de letras muertas (DLQ) para revisión manual, permitiendo que el flujo normal de transferencias continúe sin interrupciones.

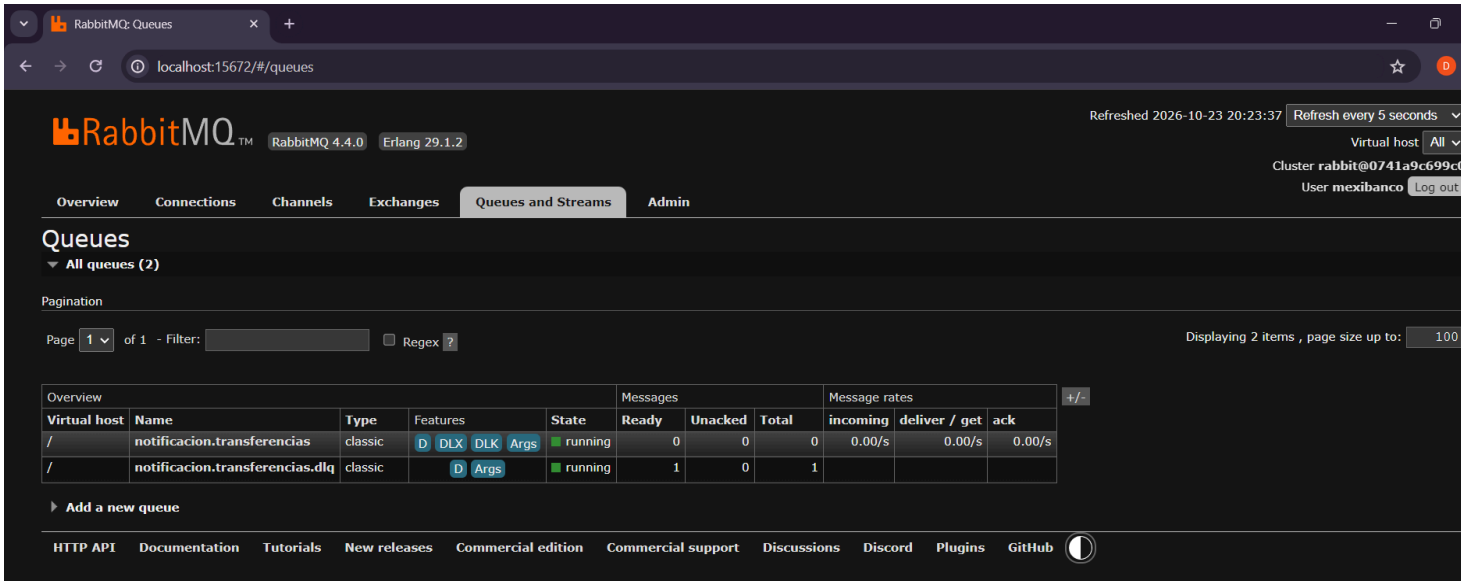


Figura 13. En Queues and Streams se ve la cola `.dlq` con 1 mensaje.



Figura 14. bandeja de notificaciones donde se ve que el aviso de tu transferencia normal (de 45 pesos) llegó con éxito

Sección 6

Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?

Si el broker está caído, la transferencia efectúa su guardado local como COMPLETADA, pero la publicación arroja una excepción al no encontrar la red. Dado que esta excepción está encapsulada en un bloque *try-catch* que solo imprime un log de error, el proceso no se detiene; el cliente recibe un 201 exitoso, pero el aviso se pierde para siempre. El *publisher confirm* no sirve de nada en este escenario porque su función exclusiva es validar que un

broker disponible recibió y guardó la información; si el broker está muerto y no hay conexión, el mecanismo de confirmación ni siquiera llega a iniciarse.

¿Por qué el ack va *después* de guardar y no antes? ¿Qué garantía obtienes con cada orden?

El ack manual se ubica estrictamente después del saveAll para garantizar que la notificación no se borre del broker hasta tener la certeza absoluta de que ya existe en nuestra base de datos. Si colocáramos el ack antes y el microservicio sufriera un corte de energía milisegundos después, RabbitMQ desecharía el mensaje asumiendo que ya terminamos, provocando pérdida definitiva de información. Al hacerlo después, la garantía cambia: ante una caída, el broker reentregará el mensaje (redelivered=true); preferimos lidiar con un posible duplicado (controlado por idempotencia) que perder un dato.

Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste?

Elección: Al implementar este modelo basado en eventos asíncronos y RabbitMQ.

Porque: Decidimos priorizar la disponibilidad de las transferencias financieras sobre la inmediatez de las notificaciones, permitiendo que el emisor concluya su flujo sin bloqueos.

Sacrifico: La consistencia estricta en tiempo real (strong consistency). Ahora, no existe garantía de que, en el instante exacto en que la pantalla le muestra "Transferencia Exitosa" al origen, el destinatario ya tenga la notificación reflejada en su bandeja, aceptando un retraso eventual